A Conservation Law for Search Work

J. R. Landers

An algorithm can shorten a search by constructing a representation that rules out whole groups of candidates. To see whether this saves work overall, charge both the construction and the remaining search. A proven lower bound constrains their sum. A model of how much structure can rule out at a given cost can then give a sharper tradeoff between the two.

1. Accounting at a proven lower bound

Fix a computational model, a class $\mathcal{A}$ of correct algorithms, and the finite set X_n of inputs of size n. For an algorithm A in the class and input x, let $D_A(x)$ count the work of discovering and constructing structure, and let $S_A(x)$ count the work of using it and completing the search. Charge each operation once and write $T_A(x)$ for the total work. For a real number u, write $[u]_+$ for its positive part.

$$T_A(x) = D_A(x) + S_A(x), \qquad [u]_+ = \max\{u, 0\}.$$

If $L(n)$ is a worst-case lower bound for the class, each algorithm A has a hard input x_A of size n. On that run, nonnegativity of structural work gives

$$T_A(x_A) \geq L(n) \quad \Longrightarrow \quad D_A(x_A) \geq [L(n) - S_A(x_A)]_+ .$$

The two costs come from the same run, although different algorithms may have different hard inputs. A runtime upper bound supplies a budget for an algorithm and leaves the lower-bound premise to be established separately.

The tradeoff under a search cap. For a nonnegative search budget s, let $\mathcal{D}_n(s)$ be the infimum of worst-case structural work over algorithms whose search stays within s on every input of size n. The infimum is $+\infty$ when no algorithm meets the cap. Since the cap holds on each eligible algorithm's hard input, the bound above gives

$$\mathcal{D}_n(s) := \inf_{\substack{A \in \mathcal{A} \\ \max_{x \in X_n} S_A(x) \leq s}} \max_{x \in X_n} D_A(x) \;\geq\; [L(n) - s]_+ .$$

Write D and S for the two costs on a run. For two allocations at the same floor, write ΔD and ΔS for the respective changes.

$$D + S = L(n) \quad \text{for both allocations} \quad \Longrightarrow \quad \Delta D = -\Delta S.$$

Above the floor, a faster search can instead consume some of the slack, the excess of $D + S$ over $L(n)$.

An exact example. Finding the maximum of n distinct keys requires $n - 1$ comparisons because every key except the maximum must lose at least once. Find the maximum of a block of k keys first, then update the incumbent while scanning the remaining keys. Varying k from 1 to n realizes every integer split of the comparison bound.

$$D = k - 1, \qquad S = n - k, \qquad D + S = n - 1.$$

2. A model for subset sum

Given n input items and a target sum, subset sum asks whether any subset sums to the target. Let N be the number of candidate subsets, C the number ruled out by the structural phase, and r the number of survivors. Consider a run that tests every survivor individually, as a no-instance requires in a model that certifies absence by checking each survivor. Let D, S, and T be the structural, search, and total work on that run.

$$N = 2^n, \qquad C + r = N.$$

Let a be a positive structural cost coefficient, β a fixed exponent in $(0, 1]$, and b a positive lower bound on the cost per survivor checked. Assume the coefficients are independent of n and the costs satisfy

$$D \geq aC^\beta, \qquad S \geq br.$$

The first inequality limits how many candidates the structural phase can rule out. The second limits how many survivors the search can check. Inverting the cost inequalities gives

$$C \leq (D/a)^{1/\beta}, \qquad r \leq S/b.$$

Adding the two bounds gives the candidate-count floor on the left below. Rearranging it using nonnegativity of D gives the structural-work bound on the right, in the same form as in Section 1.

$$(D/a)^{1/\beta} + S/b \geq 2^n \quad \Longleftrightarrow \quad \boxed{D \geq a\,[2^n - S/b]_+^\beta}.$$

If every algorithm in the class has a run satisfying both cost hypotheses, the same inequality bounds $\mathcal{D}_n(s)$ from below with s in place of S. At $\beta = 1$ the tradeoff is linear in the scaled costs. For $\beta < 1$, the minimum charge per eliminated candidate decreases as the eliminated group grows.

A lower bound on total work. Use the candidate count to express the number of survivors in terms of C. The resulting lower bound is a concave function of C on $[0, N]$, so it is at least the smaller of its endpoint values.

$$\begin{aligned} T = D + S &\geq aC^\beta + b(N - C) \\ &\geq \min\{aN^\beta, bN\} \\ &\geq \min\{a, b\}\, 2^{\beta n}. \end{aligned}$$

Here the last step uses $\beta \leq 1$. If every algorithm in the class has such a run, its worst-case cost grows at least as a positive constant times $2^{\beta n}$, written $\Omega(2^{\beta n})$. The substantive proof obligation is to establish both charges on a common hard run. The candidate count then turns those charges into a lower bound. Algorithms that can certify the survivors in bulk require a different account of residual search.

3. Deriving a floor from required progress

The direction of the argument can be reversed. Fix a model and an input size n. Suppose every correct algorithm has a run that must establish at least $K(n)$ units of some measure of progress. Suppose $g_n(D)$ and $h_n(S)$ upper-bound the progress attributable to structural work D and search

work S on that same run, with all progress accounted for. The two bounds constrain the possible cost allocations.

$$g_n(D) + h_n(S) \geq K(n), \qquad L_{\text{derived}}(n) := \inf_{\substack{D,S\geq 0 \\ g_n(D)+h_n(S)\geq K(n)}} (D+S).$$

Here $L_{\text{derived}}(n)$ is a lower bound on worst-case total work. The proof has to supply both the required progress and the limits on what the two phases can establish.

Real-valued Subset Sum in a linear decision tree. In this model a query asks for the sign of an affine linear function of the input, with at most three outcomes. Take $n \geq 2$, fix $m = \lfloor n/2 \rfloor$ weights, and let the other $p = n - m$ weights vary independently. Use the fixed weights $2^0, 2^1, \ldots, 2^{m-1}$ and set $M = 2^m$. Their subset sums are exactly the integers from 0 to $M - 1$. Set the target to $2M$ and restrict each variable weight x_j, for $1 \leq j \leq p$, to $(M, 2M + 1)$. A solution needs exactly one variable weight. The fixed weights alone fall short, while two variable weights exceed the target. Thus the input is a YES instance exactly when

$$x_j \in \{M + 1, \ldots, 2M\} \quad \text{for some } j.$$

The M target-completing values cut each coordinate into $M + 1$ open intervals. Their products form $R = (M + 1)^p$ nonempty NO regions. A transcript is the sequence of query outcomes along a root-to-leaf path. Its feasible inputs form a convex set, since each outcome imposes a linear equality or inequality. A NO transcript cannot contain points from two different regions. The segment between them would cross a YES boundary.

Let D and S now count queries in the two phases, and let q be the tree's maximum reachable depth. At least R distinct NO transcripts are necessary, and the tree has at most 3^q reachable leaves. On a run of maximum depth,

$$3^{D+S} \geq R \quad \Longrightarrow \quad D + S \geq p \log_3(2^m + 1) = \Omega(n^2).$$

Here the required progress is $\log_3 R$ units of transcript capacity. Allowing one more query, in either phase, can at most triple the number of possible transcripts. The NO regions therefore supply the floor directly. The quadratic bound is classical. Dobkin and Lipton counted connected components, and Ben-Or extended the bound to algebraic computation trees [1].

4. Transferring a floor to integer Subset Sum

For n positive integer weights and positive integer target B, a different argument gives a bound conditional on the Strong Exponential Time Hypothesis (SETH). In the form used here, SETH says that for every fixed $0 < \rho < 1$ there is a clause width k for which satisfiability of k-CNF formulas on v variables admits no worst-case algorithm in $O(2^{(1-\rho)v})$ time. We use the reduction of Abboud, Bringmann, Hermelin, and Shabtay [2].

For any fixed $\eta > 0$, the reduction produces at most $2^{\eta v}$ Subset Sum instances, each with $O(v)$ weights and a target at most $2^{(1+2\eta)v}$. The hidden constant in the item count can depend on k and η. The formula is satisfiable exactly when some instance has a solution. The reduction costs $2^{\eta v}$ times a polynomial in v.

Suppose an integer Subset Sum solver has a uniform total-time bound

$$D + S = O\Big(B^{1-\epsilon} 2^{o(n)}\Big) \qquad \text{for a fixed } 0 < \epsilon < 1.$$

Here $o(n)$ denotes an exponent whose ratio to n tends to zero. For fixed k and η, the bound $n = O(v)$ makes that exponent $o(v)$ on the reduced instances. Running the solver on all instances would then take time bounded above, up to a constant, by

$$2^{\eta v}\, 2^{(1+2\eta)(1-\epsilon)v+o(v)} = 2^{[\eta+(1+2\eta)(1-\epsilon)]v+o(v)}.$$

Choose $\eta = \epsilon/8$. The bracketed coefficient is at most $1 - \epsilon + 3\eta = 1 - 5\epsilon/8$. For sufficiently large v, the $o(v)$ term is at most $\epsilon v/8$, and the reduction time has a smaller exponent as well. The combined algorithm would solve k-SAT in $O(2^{(1-\epsilon/2)v})$ time for every fixed k. Taking $\rho = \epsilon/2$ gives a contradiction under SETH.

In the decision tree, the floor comes from how many separated NO regions the algorithm must distinguish. In the integer model, the reduction transfers a limit on the total budget from SAT. An unconditional proof by the first route would require a measure of progress whose capacity bound survives integer operations that can resolve many cases at once.

5. Practical use: deciding whether to prune

Consider a Subset Sum workload on which the baseline scans all N candidates. Let $b > 0$ be the time per check. A proposed pruning step takes time D, eliminates C candidates, and leaves the cost of each remaining check unchanged. Charge all pruning overhead to D. Write T_{base} for the baseline time and T_{prune} for the time with pruning.

$$T_{\text{base}} = bN, \qquad T_{\text{prune}} = D + b(N - C).$$

The pruning step saves time exactly when the checks it avoids save more time than the step costs.

$$T_{\text{prune}} < T_{\text{base}} \iff bC > D \iff C > D/b.$$

A proven upper bound on the number of checks avoided can rule out the proposal before implementation. If the upper bound is at or below D/b, the step cannot speed up the scan under this cost model.

Worked example. Suppose a baseline scan checks one million candidates, each check takes one microsecond, and the pruning step costs 200 milliseconds.

$$T_{\text{base}} = 1\,\text{s}, \qquad D/b = \frac{200\,\text{ms}}{1\,\mu\text{s}} = 200{,}000 \text{ checks}.$$

Eliminating exactly 200,000 checks breaks even. Suppose the proposed representation can eliminate at most 150,000 checks on this workload. It leaves at least 850,000 checks, so

$$T_{\text{prune}} \geq 200\,\text{ms} + 850\,\text{ms} = 1.05\,\text{s}.$$

The proposed step would make this scan at least 50 milliseconds slower. If the bound on eliminated checks exceeds the break-even threshold, measure construction and residual search together against the baseline. The threshold tells us how much work a representation must save to justify its cost.

References

[1] M. Ben-Or, *Lower Bounds for Algebraic Computation Trees (Preliminary Report)*, Proc. STOC, 80–86, 1983. See Example 6(a) and its attribution to Dobkin and Lipton. DOI

[2] A. Abboud, K. Bringmann, D. Hermelin, and D. Shabtay, *SETH-based Lower Bounds for Subset Sum and Bicriteria Path*, ACM Transactions on Algorithms, 18(1), 2022. DOI